

DEEP LEARNING SEMESTER GANJIL 2024 - 2025
DOG BREED CLASSIFICATION



LAPORAN PROYEK

Muhammad Daffa' Fisabilillah	5024211006
Arsenius Audley Wahyu Djatmiko	5024211030

DOSEN PENGAMPU

Dr. I Ketut Eddy Purnama, S.T., M.T.

DEPARTEMEN TEKNIK KOMPUTER
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA
CERDAS INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA

BACKGROUND

Deep learning telah menjadi salah satu pendekatan utama dalam penyelesaian berbagai masalah dalam kecerdasan buatan, terutama dalam pengolahan citra dan pengenalan objek. Salah satu aplikasi utama deep learning adalah **multi-class classification**, yang memungkinkan komputer untuk mengklasifikasikan data atau objek ke dalam beberapa kategori, seperti mengidentifikasi berbagai jenis hewan dari gambar yang diberikan. Penerapan teknologi ini dalam aplikasi web dan mobile sangat penting, karena memberikan kemudahan bagi pengguna untuk memanfaatkan model klasifikasi dalam kehidupan sehari-hari, baik melalui perangkat dengan sumber daya besar seperti komputer desktop atau perangkat mobile yang lebih terbatas.

Pada proyek ini, kami memilih untuk mengembangkan klasifikasi breed anjing, yang merupakan topik relevan bagi pecinta anjing dan aplikasi yang digunakan oleh shelter hewan. Aplikasi yang dapat mengenali berbagai breed anjing melalui gambar dapat membantu para pecinta anjing dalam mengetahui lebih banyak tentang ras tertentu dan memudahkan shelter hewan dalam mengelompokkan anjing yang mereka tampung. Selain itu, aplikasi semacam ini memiliki potensi untuk diterapkan dalam berbagai bidang, seperti pelatihan anjing, konservasi hewan, dan pengenalan gambar hewan untuk keperluan lainnya.

Proyek ini juga memiliki signifikansi dalam konteks pengembangan model deep learning dari awal (scratch). Membangun model dari awal memberikan pemahaman yang lebih mendalam mengenai prinsip-prinsip dasar deep learning dan tantangan yang dihadapi dalam pengembangan model klasifikasi gambar yang efektif. Meskipun transfer learning dapat memberikan hasil yang lebih cepat dan lebih akurat dalam aplikasi nyata, membangun model dari awal memungkinkan kami untuk lebih memahami proses pelatihan model, optimasi performa, dan penyelesaian masalah yang mungkin muncul saat mengintegrasikan model ke dalam aplikasi web dan mobile. Dengan pendekatan ini, kami berharap dapat memperoleh wawasan lebih dalam mengenai pembuatan model deep learning dan kemampuan untuk mengatasi tantangan praktis dalam penerapannya.

PROBLEMS

Pada proyek ini, terdapat beberapa masalah utama yang harus dihadapi selama pengembangan dan implementasi model deep learning untuk klasifikasi breed anjing. Pertama, tantangan dalam pembuatan model deep learning dari awal (scratch). Meskipun deep learning memiliki potensi besar untuk melakukan klasifikasi gambar, membangun model dari awal memerlukan pemahaman mendalam tentang arsitektur jaringan, pemilihan hyperparameters, serta cara mengatasi masalah seperti overfitting dan underfitting. Proses ini jauh lebih kompleks dibandingkan dengan menggunakan teknik seperti transfer learning, yang memungkinkan penggunaan model yang sudah dilatih sebelumnya, sehingga mempercepat pengembangan dan meningkatkan akurasi model.

Masalah berikutnya adalah terkait dengan dataset yang terbatas dan keragamannya. Dalam kasus klasifikasi breed anjing, dataset yang digunakan mungkin tidak cukup beragam untuk menangani variasi gambar yang dapat muncul, seperti pencahayaan yang buruk, sudut pengambilan gambar yang berbeda, atau kualitas gambar yang rendah. Hal ini dapat menyebabkan model kesulitan dalam mengenali breed anjing dengan akurat, terutama dalam kondisi dunia nyata. Selain itu, kualitas gambar dan jumlah data yang terbatas dapat mempengaruhi performa model, membuatnya lebih rentan terhadap kesalahan klasifikasi.

Selanjutnya, permasalahan dalam integrasi model ke dalam aplikasi web dan mobile. Mengonversi dan mengintegrasikan model deep learning ke dalam aplikasi web menggunakan TensorFlow (.keras) dan mobile menggunakan PyTorch (dikonversi ke TFLite) menghadirkan tantangan tersendiri. Proses konversi model dan optimisasi agar berjalan dengan baik di kedua platform tersebut membutuhkan perhatian ekstra, terutama dalam mengurangi ukuran model agar tetap efisien tanpa mengurangi akurasi. Selain itu, perbedaan antara platform web dan mobile juga memperkenalkan masalah dalam hal kecepatan eksekusi, pengelolaan sumber daya, dan kompatibilitas dengan perangkat keras yang ada.

Terakhir, perbandingan hasil antara model yang dibangun dari awal (scratch) dengan model berbasis transfer learning juga menjadi masalah penting. Meskipun model yang dibangun dari awal memberikan pemahaman yang lebih baik tentang mekanisme deep learning, performa model ini sering kali lebih rendah dibandingkan dengan menggunakan transfer learning, yang dapat mencapai akurasi yang lebih tinggi dalam waktu lebih singkat. Hal ini menimbulkan pertanyaan mengenai apakah membangun model dari awal adalah pendekatan yang paling efisien dan optimal, atau jika model transfer learning lebih tepat digunakan dalam aplikasi nyata.

GOAL

Tujuan utama dari proyek ini adalah untuk membangun dan melatih model deep learning dari awal (scratch) yang dapat mengklasifikasikan 10 breed anjing dengan akurasi yang optimal. Kami bertujuan untuk memahami dan mengimplementasikan seluruh proses pengembangan model, mulai dari pemilihan arsitektur jaringan, pengumpulan dan persiapan dataset, hingga proses pelatihan dan evaluasi model. Dengan membangun model dari awal, kami berharap dapat memperoleh pemahaman yang lebih dalam tentang mekanisme dasar deep learning dan tantangan yang dihadapi dalam pengembangan model klasifikasi gambar.

Selain itu, tujuan lain dari proyek ini adalah untuk mengintegrasikan model yang telah dilatih ke dalam aplikasi web dan mobile. Model yang dikembangkan dalam framework TensorFlow akan diintegrasikan ke dalam aplikasi web, sementara model yang dikembangkan menggunakan PyTorch akan dikonversi ke format TFLite untuk dapat dioperasikan pada aplikasi mobile. Integrasi ini bertujuan untuk memberikan pengalaman pengguna yang lebih interaktif dan praktis, baik melalui perangkat desktop maupun mobile.

Hasil yang diharapkan dari proyek ini adalah untuk memahami langkah-langkah mendalam dalam pengembangan model deep learning dari awal, termasuk bagaimana melakukan optimasi model, mengatasi masalah yang muncul selama pelatihan, serta menyelesaikan tantangan teknis yang berkaitan dengan konversi dan integrasi model ke dalam aplikasi. Selain itu, proyek ini juga bertujuan untuk membandingkan performa model yang dibangun dari awal dengan model yang menggunakan pendekatan transfer learning, sehingga dapat mengevaluasi kelebihan dan kekurangan dari kedua pendekatan dalam konteks aplikasi klasifikasi breed anjing.

METHOD

Pada proyek ini, dikembangkan sistem klasifikasi untuk mengenali 10 jenis anjing dengan memanfaatkan framework deep learning PyTorch. Proses pembuatan mencakup beberapa tahapan penting, yaitu desain arsitektur model, preprocessing data, pelatihan model dengan teknik augmentasi data, evaluasi model, serta penyimpanan dan konversi model ke format TFLite untuk MobileApp dan keras untuk WebApp agar dapat diimplementasikan pada perangkat mobile. Selain itu Penulis juga membandingkan hasil ketika menggunakan sebuah model transfer learning MobileNet dan ResNetV2 Bagian ini akan mengulas secara mendetail tahapan-tahapan tersebut beserta penjelasan kode yang telah diberikan.

1. Dataset

Dataset yang digunakan dalam proyek ini berasal dari **Kaggle Stanford Dog Breed Dataset**, yang terdiri dari gambar-gambar anjing yang telah diberi label sesuai dengan 10 breed anjing yang berbeda. Gambar-gambar tersebut telah diproses melalui beberapa tahapan **preprocessing** untuk memastikan kualitas data yang optimal sebelum digunakan dalam pelatihan model. Tahapan preprocessing yang dilakukan meliputi:

- **Resizing Gambar:** Gambar-gambar dalam dataset memiliki ukuran yang bervariasi, sehingga untuk memastikan konsistensi dan efisiensi dalam pemrosesan, gambar-gambar tersebut diubah ukurannya. Untuk model **scratch**, ukuran gambar diubah menjadi **177x177 piksel**, sementara untuk model **transfer learning**, gambar diubah menjadi **224x224 piksel**. Ukuran yang lebih besar untuk transfer learning membantu model pre-trained untuk mengenali fitur gambar dengan lebih baik.
- **Augmentasi Data:** Untuk meningkatkan keragaman data dan mengurangi risiko **overfitting**, augmentasi data dilakukan pada dataset pelatihan.

Teknik augmentasi yang diterapkan untuk model **WebApp** dilakukan menggunakan **ImageDataGenerator** di TensorFlow, dengan konfigurasi sebagai berikut:

```
train_datagen = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=20,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    brightness_range=[0.8, 1.2],  
    zoom_range=0.2,  
    horizontal_flip=True,
```

```

        validation_split=0.2,

        shuffle=True,  # Ensure the data is shuffled for each epoch

    preprocessing_function=tf.keras.applications.imagenet_utils.prepro
    cess_input
)

```

Sementara **Aplikasi Mobile**, Augmentasi dilakukan menggunakan `transforms.Compose` dari PyTorch dengan konfigurasi berikut:

```

transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),  # Membalik gambar
    secara horizontal dengan probabilitas 50%
    transforms.RandomRotation(10),           # Rotasi acak hingga
    10 derajat
    transforms.ColorJitter(                   # Penyesuaian
    pencahayaan, kontras, dan saturasi
        brightness=0.1,
        contrast=0.1,
        saturation=0.1
    ),
    transforms.ToTensor()                     # Mengubah gambar
    menjadi tensor
])

```

- **Normalisasi:** Semua gambar yang diproses dinormalisasi dengan membagi nilai pixel dengan 255, sehingga nilai pixel gambar berada dalam rentang [0, 1]. Ini penting untuk mempercepat konvergensi selama pelatihan model dan menghindari perbedaan skala yang besar antar fitur. Teknik normalisasi ini dilakukan dengan parameter `rescale=1./255` dalam `ImageDataGenerator`.
- **Stratified Split:** Pada **Aplikasi Mobile**, Dataset dibagi menggunakan `stratified split` untuk memastikan distribusi kelas yang seimbang di subset pelatihan dan validasi:

```
sss = StratifiedShuffleSplit(n_splits=1, test_size=valid_size,
                             random_state=SEED)

train_idx, val_idx = next(sss.split(np.zeros(len(labels)),
                                labels))
```

2. Arsitektur Model

Pada proyek ini, dua jenis arsitektur neural network digunakan: satu dibangun dari awal (scratch) dan satu menggunakan **transfer learning** untuk setiap aplikasi (web dan mobile).

2.1 Arsitektur Model untuk Web App:

Model Scratch Tensorflow:

Model ini terdiri dari beberapa layer convolutional dan fully connected, dengan penekanan pada ekstraksi fitur penting dari gambar menggunakan layer konvolusional dan pengklasifikasian dengan layer fully connected. Berikut adalah penjelasan rinci mengenai arsitektur model ini.

- **Convolutional Layers (Ekstraksi Fitur)**

Arsitektur model dimulai dengan beberapa layer konvolusional yang berfungsi untuk mengekstraksi fitur-fitur penting dari gambar. Pada setiap blok konvolusional, terdapat beberapa komponen:

- ❖ **Layer Konvolusi (Conv2D):** Layer ini berfungsi untuk mengkonvolusi gambar input dengan filter untuk mengekstrak fitur. Setiap layer menggunakan kernel berukuran kecil (misalnya, 3x3), yang mengarah pada ekstraksi fitur lokal.
- ❖ **Batch Normalization:** Digunakan setelah layer konvolusional untuk menormalkan output layer dan mempercepat proses pelatihan dengan mengurangi masalah vanishing/exploding gradients.
- ❖ **ReLU (Rectified Linear Unit):** Fungsi aktivasi ReLU digunakan setelah konvolusi untuk meningkatkan non-linearitas dalam model, memungkinkan model belajar fitur yang lebih kompleks.
- ❖ **MaxPooling:** Pooling dilakukan untuk mereduksi dimensi spasial dan mempertahankan informasi penting sambil mengurangi komputasi.

Berikut adalah potongan kode yang menunjukkan layer-layer konvolusional:

```

self.feet = tf.keras.Sequential([
    ConvBlock(32, kernel_size=7, strides=2, padding='same'), #
    Ekstraksi fitur awal
    layers.MaxPool2D(pool_size=3, strides=2, padding='same') #
    Pengurangan dimensi spasial
])

self.body = tf.keras.Sequential([
    ConvBlock(64), # Blok konvolusi untuk fitur level lebih
    tinggi
    ResidualBlock(64), # Blok residual untuk mengatasi degradasi
    performa
    layers.MaxPool2D(pool_size=2), # Pooling untuk downsampling
    layers.Dropout(0.1), # Regularisasi untuk menghindari
    overfitting

    ConvBlock(128), # Blok konvolusi lebih banyak untuk ekstraksi
    fitur tingkat lanjut
    ResidualBlock(128),
    layers.MaxPool2D(pool_size=2),
    layers.Dropout(0.1),

    ConvBlock(256), # Fitur lebih kompleks
    ResidualBlock(256),
    layers.MaxPool2D(pool_size=2),
    layers.Dropout(0.1)
])

```

Dalam bagian ini, model menggunakan **ConvBlock**, yang terdiri dari layer konvolusi, batch normalization, dan fungsi aktivasi ReLU. Setelah itu, setiap blok diikuti oleh MaxPooling untuk downsampling.

- **Residual Blocks**

Residual blocks digunakan untuk meningkatkan performa dengan memperkenalkan shortcut connections yang memungkinkan aliran gradien yang lebih baik selama pelatihan. Setiap **ResidualBlock** terdiri dari dua lapisan konvolusi yang dihubungkan dengan penjumlahan residual ke output.

```

class ResidualBlock(layers.Layer):
    def __init__(self, filters):
        super().__init__()
        self.conv1 = ConvBlock(filters)

```



```

self.conv2 = ConvBlock(filters)
self.relu = layers.ReLU()

def call(self, x):
    residual = x # Shortcut connection
    out = self.conv1(x)
    out = self.conv2(out)
    out = out + residual # Menambahkan residual ke output
    return self.relu(out)

```

Residual blocks membantu model untuk belajar fitur yang lebih dalam dengan mempermudah aliran gradien, yang pada gilirannya membantu mencegah masalah vanishing gradients dan meningkatkan performa pada layer yang lebih dalam.

- **Fully Connected Layers (Klasifikasi)**

Setelah ekstraksi fitur menggunakan layer konvolusional, model akan mengalirkan fitur yang telah diproses melalui layer fully connected untuk pengklasifikasian.

- ❖ **GlobalAveragePooling2D:** Mengurangi dimensi data menjadi satu nilai per kanal (fitur) dengan mengambil rata-rata di seluruh spatial grid, yang mengurangi kemungkinan overfitting dan meningkatkan generalisasi.
- ❖ **Dense Layer:** Layer ini menghubungkan semua fitur yang dipelajari dari layer konvolusional ke unit keluaran akhir. Fungsi aktivasi ReLU digunakan untuk meningkatkan non-linearitas, sementara output dari layer terakhir menggunakan softmax untuk multi-class classification.

Berikut adalah potongan kode untuk bagian head dari model:

```

self.head = tf.keras.Sequential([
    layers.GlobalAveragePooling2D(), # Mereduksi dimensi spasial
    layers.Flatten(), # Meratakan output menjadi vektor
    layers.Dense(num_classes) # Klasifikasi menggunakan dense
layer
])

```

- **Regularisasi dan Optimasi**

Untuk mencegah overfitting, beberapa teknik regularisasi digunakan:

- ❖ **Dropout:** Digunakan di beberapa layer untuk mengurangi overfitting dengan mematikan sejumlah unit secara acak selama pelatihan.
- ❖ **Batch Normalization:** Digunakan setelah layer konvolusi untuk mempercepat konvergensi dan mengurangi overfitting dengan menormalkan setiap batch.

- ❖ **Weight Decay:** Digunakan untuk mengurangi ukuran bobot model selama pelatihan, mencegah overfitting dengan memberikan penalti pada bobot yang terlalu besar.

Model Transfer Learning ResNet50V2:

Model ini menggunakan pre-trained model **ResNet50V2** yang diadaptasi untuk klasifikasi breed anjing. Transfer learning memungkinkan penggunaan model yang sudah dilatih pada dataset besar (seperti ImageNet), yang sudah belajar untuk mengenali pola-pola umum dalam gambar. Dalam kasus ini, model tersebut akan disesuaikan untuk mengklasifikasikan 10 jenis breed anjing. Dengan pendekatan ini, kita dapat memanfaatkan pengetahuan yang sudah ada dan mempercepat proses pelatihan serta meningkatkan akurasi dengan sedikit penyesuaian (fine-tuning) pada lapisan terakhir.

Base Model (Pre-trained ResNet50V2):

- Menggunakan **ResNet50V2** yang sudah dilatih dengan dataset ImageNet. Model ini tidak mencakup lapisan klasifikasi (top layer) karena kita hanya menggunakan fitur-fitur yang telah dipelajari dari dataset tersebut.
- Model ini mengimplementasikan residual blocks yang memungkinkan pelatihan model lebih dalam dengan mengatasi masalah degradasi.

Global Average Pooling:

- Setelah melalui pre-trained base model, output dari ResNet50V2 yang berbentuk peta fitur dilanjutkan ke **GlobalAveragePooling2D** untuk mengurangi dimensi fitur dan mengubah peta fitur menjadi vektor fitur satu dimensi yang lebih cocok untuk klasifikasi.

Lapisan Fully Connected (Dense):

- **Dense layer pertama** dengan 1024 neuron, dilengkapi dengan aktivasi **ReLU** untuk menambah non-linearitas model.
- **BatchNormalization** digunakan untuk mempercepat konvergensi dan mengurangi overfitting.
- **Dropout** digunakan pada lapisan kedua dan ketiga untuk menghindari overfitting selama pelatihan.

Lapisan Klasifikasi:

- Lapisan output menggunakan **Dense layer** dengan jumlah neuron sebanyak jumlah kelas yang ada, yaitu 10 untuk 10 breed anjing. Fungsi aktivasi yang digunakan adalah **softmax** untuk mengklasifikasikan ke dalam beberapa kelas.

Fine-tuning:

- Selama fine-tuning, lapisan-lapisan terakhir dari model ResNet50V2 di-unfreeze agar dapat belajar kembali, namun dengan learning rate yang lebih kecil dibandingkan dengan tahap pelatihan awal.

```

def create_resnet_model(num_classes=10, fine_tune_layers=0):
    """
    Create a ResNet50V2 model with custom top layers for
    classification
    """
    # Load pre-trained ResNet50V2 (using V2 for better
    performance)
    base_model = tf.keras.applications.ResNet50V2(
        include_top=False,
        weights='imagenet',
        input_shape=(224, 224, 3) # Standard ImageNet size
    )

    # Initially freeze all layers
    base_model.trainable = False

    # Create custom top layers
    inputs = base_model.input
    x = base_model.output
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dense(1024, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.5)(x)
    x = layers.Dense(512, activation='relu')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(num_classes)(x)

    model = Model(inputs=inputs, outputs=outputs)

    return model, base_model

```

2.2 Arsitektur Model untuk Mobile App:

Model Scratch Pytorch:

Arsitektur model yang digunakan pada aplikasi mobile ini dinamakan *CustomCNN*, sebuah *Convolutional Neural Network (CNN)* yang dirancang untuk kebutuhan klasifikasi 10 kelas dengan efisiensi tinggi. Arsitektur model ini terbagi menjadi tiga bagian utama: **Feet**, **Body**, dan **Head**, masing-masing dirancang dengan fungsi spesifik untuk menangkap, memproses, dan mengklasifikasikan fitur dari gambar masukan.

- **Feet (Ekstraksi Awal Fitur)**

Bagian pertama, **Feet**, bertugas mengekstrak fitur awal dari gambar masukan. Gambar dengan tiga saluran warna (RGB) diproses melalui lapisan *convolutional* pertama menggunakan kernel berukuran besar (7x7) dengan stride 2, yang berfungsi menangkap pola global pada gambar. Setelahnya, dimensi gambar dikurangi menggunakan lapisan **MaxPool2d**, yang membantu menjaga efisiensi komputasi tanpa menghilangkan informasi penting. Berikut adalah implementasi kode bagian Feet:

```
self.feet = nn.Sequential(  
    ConvBlock(3, 32, kernel_size=7, stride=2, padding=3),  
    nn.MaxPool2d(kernel_size=3, stride=2, padding=1)  
)
```

- **Body (Pemrosesan Fitur)**

Setelah fitur awal diekstraksi, bagian **Body** memproses fitur tersebut secara hierarkis melalui beberapa tahap pemrosesan. Setiap tahap terdiri dari blok konvolusi (*ConvBlock*), blok residual (*ResidualBlock*), *pooling*, dan *dropout*. Blok residual menggunakan koneksi shortcut untuk memastikan gradien tetap mengalir selama pelatihan, sehingga membantu mencegah masalah *vanishing gradient*. Lapisan *MaxPool2d* digunakan untuk mengurangi dimensi spasial lebih lanjut di setiap tahap, sedangkan dropout dengan probabilitas rendah (0.1) diterapkan untuk meningkatkan kemampuan generalisasi model dan mencegah overfitting. Berikut implementasi kode bagian Body:

```
self.body = nn.Sequential(  
    # Stage 1  
    ConvBlock(32, 64),  
    ResidualBlock(64),  
    nn.MaxPool2d(2),  
    nn.Dropout2d(0.1),  
  
    # Stage 2  
    ConvBlock(64, 128),  
    ResidualBlock(128),  
    nn.MaxPool2d(2),  
    nn.Dropout2d(0.1),  
  
    # Stage 3  
    ConvBlock(128, 256),  
    ResidualBlock(256),  
    nn.MaxPool2d(2),  
    nn.Dropout2d(0.1)  
)
```

- **Head (Klasifikasi)**

Bagian terakhir, **Head**, bertugas mengubah representasi fitur yang telah diproses menjadi prediksi kelas. Di bagian ini, digunakan lapisan `AdaptiveAvgPool2d` untuk memastikan ukuran keluaran tetap, terlepas dari resolusi gambar masukan, sehingga memungkinkan fleksibilitas model untuk digunakan pada perangkat mobile dengan resolusi gambar yang bervariasi. Setelah fitur diratakan menggunakan `Flatten`, sebuah lapisan *fully connected* (Linear) memetakan representasi fitur ke dalam 10 kelas dengan keluaran berupa probabilitas untuk masing-masing kelas. Berikut adalah implementasi bagian Head:

```
self.head = nn.Sequential(
    nn.AdaptiveAvgPool2d(1),
    nn.Flatten(),
    nn.Linear(256, num_classes) # Langsung ke jumlah kelas
)
```

- **Inisialisasi Bobot**

Seluruh arsitektur model ini diinisialisasi menggunakan metode *Kaiming Initialization*, yang dirancang khusus untuk aktivasi ReLU agar bobot tetap stabil selama pelatihan. Ini memastikan distribusi gradien yang optimal dan mencegah gradien menjadi terlalu besar atau terlalu kecil saat melewati banyak lapisan. Berikut adalah kode inisialisasi bobotnya:

```
def _initialize_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out',
nonlinearity='relu')
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.constant_(m.weight, 1)
            nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, std=0.01)
            nn.init.constant_(m.bias, 0)
```

Dengan kombinasi desain modular ini, *CustomCNN* menjadi model yang ringan namun tetap mampu menangkap fitur kompleks dari gambar untuk menghasilkan klasifikasi dengan akurasi tinggi. Penerapan blok residual, dropout, dan inisialisasi bobot memastikan pelatihan yang stabil dan performa generalisasi yang baik, membuatnya ideal untuk perangkat mobile dengan sumber daya terbatas.

Model Transfer Learning MobileNetV2:

Model klasifikasi berbasis MobileNetV2 dirancang menggunakan arsitektur transfer learning yang efisien untuk menangani tugas klasifikasi dengan sumber daya perangkat yang terbatas. MobileNetV2 digunakan sebagai backbone karena ukurannya yang ringan dan performa akurasi yang tinggi. Model ini memanfaatkan bobot yang telah dilatih sebelumnya pada dataset ImageNet dan menyesuaikannya dengan kebutuhan klasifikasi 10 kelas melalui pengembangan head model.

Komponen Utama Arsitektur Model

- **Backbone MobileNetV2:**
 - *Input Shape:* (224, 224, 3), disesuaikan dengan dimensi citra input.
 - *Pre-trained Weights:* Menggunakan bobot dari ImageNet.
 - *Include Top:* False, untuk menghilangkan lapisan fully connected asli.
 - *Frozen Layers:* Lapisan backbone dibekukan untuk menjaga bobot yang sudah dilatih sebelumnya.
- **Head Model:**
 - Lapisan konvolusi tambahan dengan filter untuk ekstraksi fitur.
 - Lapisan Dropout untuk mencegah overfitting.
 - Lapisan GlobalAveragePooling2D untuk meratakan dimensi spasial menjadi vektor 1D.
 - Lapisan Dense dengan aktivasi softmax untuk memprediksi probabilitas dari 10 kelas.

```
import tensorflow as tf

# Inisialisasi MobileNetV2 sebagai backbone
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)
base_model.trainable = False # Membekukan lapisan backbone

# Menambahkan head model
model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.Conv2D(32, 3, activation='relu'), # Ekstraksi fitur tambahan
    tf.keras.layers.Dropout(0.2), # Regularisasi untuk mencegah overfitting
    tf.keras.layers.GlobalAveragePooling2D(), # Meratakan dimensi spasial
    tf.keras.layers.Dense(10, activation='softmax') # Prediksi probabilitas 10 kelas
])

# Kompilasi model
model.compile(

optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),

loss=tf.keras.losses.CategoricalCrossentropy(from_logits=False),
```

```
        metrics=['accuracy']
    )

    model.summary()
```

3. Training

Proses pelatihan dilakukan dengan mengikuti langkah-langkah berikut:

Hyperparameters:

- **Optimizer:** Menggunakan **Adam optimizer** karena kemampuannya dalam menangani berbagai jenis masalah dan konvergensi yang stabil.

Aplikasi Mobile:

```
optimizer = optim.AdamW(
    model.parameters(), lr=LEARNING_RATE,
    weight_decay=WEIGHT_DECAY
)
```

Web:

```
model.compile(
    optimizer=tf.keras.optimizers.AdamW(
        learning_rate=config['learning_rate'],
        weight_decay=config['weight_decay'],
        clipnorm=1.0
    ),
```

- **Loss:** Fungsi loss yang digunakan adalah `CrossEntropyLoss` dengan label smoothing untuk menangani ketidakseimbangan kelas secara halus:

Aplikasi Mobile:

```
criterion = nn.CrossEntropyLoss(label_smoothing=0.02)
```

Web:

```
loss=tf.keras.losses.CategoricalCrossentropy(from_logits
=True, label_smoothing=0.1),
    metrics=['accuracy']
)
```

- **Learning Rate:** Learning rate diatur pada 0.0001 untuk model scratch dan 0.00001 untuk model transfer learning untuk memastikan model tidak terlalu cepat belajar dan melompat ke solusi yang tidak optimal.
- **Batch Size:** Batch size ditentukan sebesar 32 untuk memastikan efisiensi selama pelatihan, sambil menghindari penggunaan memori yang berlebihan.
- **Jumlah Epoch:** Model dilatih selama 30-50 epoch, dengan **early stopping** untuk mencegah overfitting dan menghemat waktu pelatihan.

Evaluasi Model: Untuk mengevaluasi kinerja model, dilakukan **train-test split** dengan rasio 80:20, serta **cross-validation** untuk memastikan model tidak overfit dan dapat menggeneralisasi dengan baik pada data yang tidak terlihat sebelumnya.

4. Integrasi ke Aplikasi

Web App

Model yang dibangun menggunakan TensorFlow (.keras) akan diintegrasikan ke dalam aplikasi web menggunakan framework Flask. Flask akan bertindak sebagai backend untuk menerima permintaan gambar, memproses gambar menggunakan model TensorFlow, dan mengirimkan hasil prediksi kembali ke frontend aplikasi.

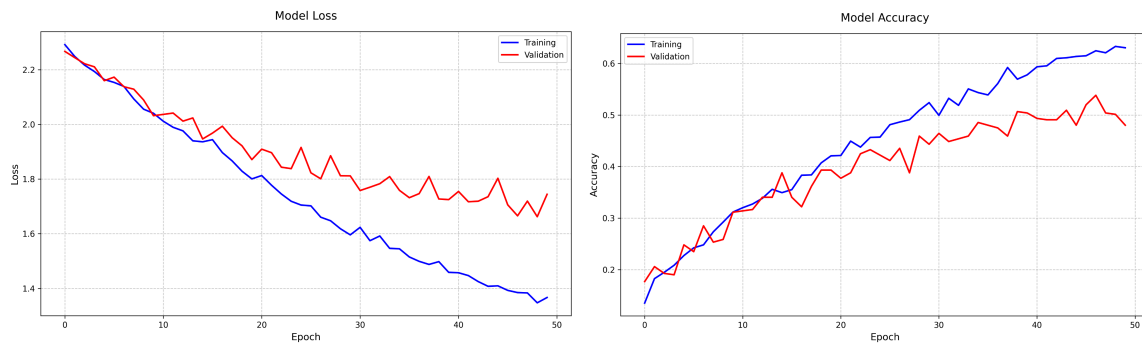
Mobile App:

Untuk memungkinkan implementasi di aplikasi mobile, model yang dilatih menggunakan PyTorch dikonversi ke format TensorFlow Lite (TFLite) agar dapat dijalankan secara efisien pada perangkat Android atau iOS. Proses ini dilakukan melalui langkah-langkah sistematis, dimulai dengan mengekspor model dari PyTorch ke format ONNX (Open Neural Network Exchange) sebagai perantara. Format ONNX digunakan untuk memastikan interoperabilitas antar-framework deep learning. Model kemudian dikonversi dari ONNX ke TensorFlow menggunakan *ONNX-TensorFlow Converter*, yang menghasilkan model dalam format yang dapat diproses lebih lanjut untuk menghasilkan file TFLite.

RESULT

Model Web App:

Model 1 Scratch PyTorch



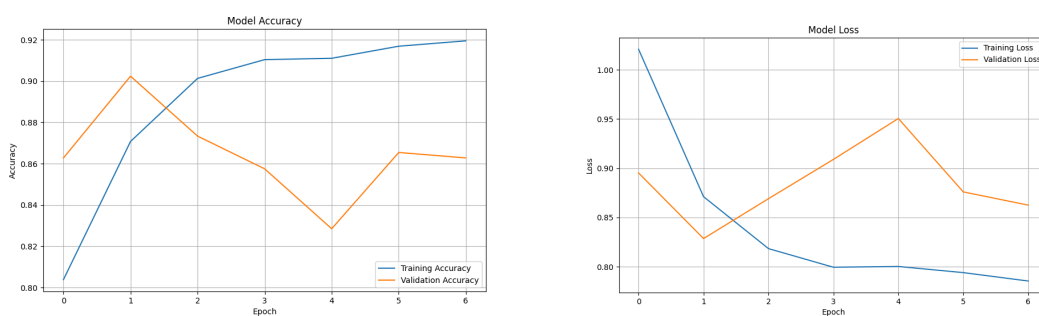
1. Akurasi Model:

- Training accuracy meningkat secara konsisten dari sekitar 0.15 hingga 0.63 (63%) pada epoch ke-50
- Validation accuracy juga meningkat tapi lebih rendah, mencapai sekitar 0.48 (48%) di akhir training
- Terdapat gap yang semakin melebar antara training dan validation accuracy setelah epoch ke-30, yang mengindikasikan overfitting

2. Loss Model:

- Training loss menurun secara konsisten dari 2.3 ke 1.4
- Validation loss juga menurun tapi tidak sebaik training loss, berakhir di sekitar 1.7
- Gap antara training dan validation loss yang semakin lebar juga mengkonfirmasi adanya overfitting

Model 2 ResNetV2



1. Akurasi Model:

- Training accuracy meningkat signifikan dari 80% menjadi 92% dalam 6 epoch
- Validation accuracy mencapai puncak sekitar 90% pada epoch ke-1, namun kemudian menurun dan berfluktuasi, akhirnya stabil di sekitar 86%
- Terjadi gap yang semakin besar antara training dan validation accuracy setelah epoch ke-1

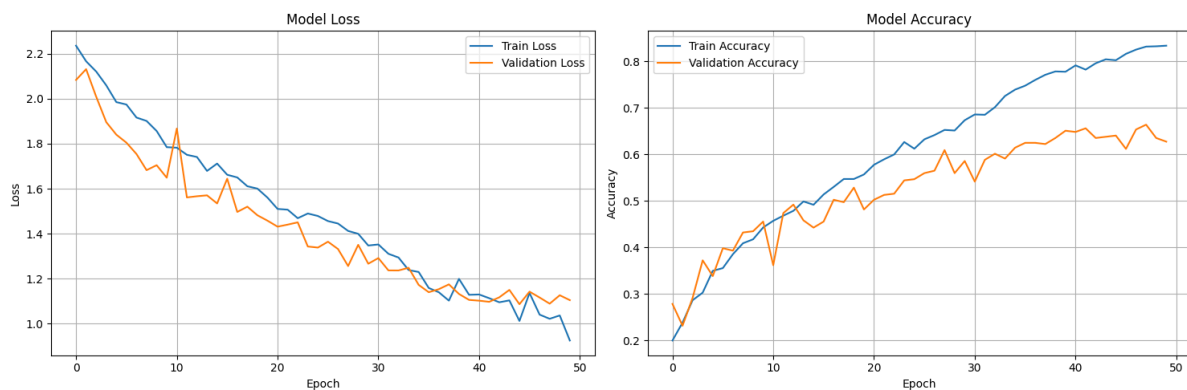
2. Loss Model:

- Training loss menurun tajam dari 1.0 ke 0.8

- Validation loss berfluktuasi, dengan peningkatan setelah epoch ke-1 dan mencapai puncak sekitar 0.95 pada epoch ke-4
- Pola validation loss yang meningkat sementara training loss menurun adalah indikasi klasik overfitting

Model Mobile App:

Model 1 Scratch PyTorch



1. Analisis Training dan Validation Loss

Pada grafik "Model Loss" (kiri), terlihat pola penurunan loss baik pada data training maupun validasi seiring bertambahnya jumlah epoch:

1. **Awal Pelatihan:** Nilai loss pada data training dan validasi memulai pelatihan di angka yang tinggi (sekitar 2.2), yang menunjukkan bahwa model belum memiliki kemampuan untuk memprediksi dengan akurat.
2. **Penurunan Loss:** Dengan bertambahnya epoch, training loss secara konsisten menurun hingga mencapai nilai mendekati 1.0 pada akhir pelatihan. Hal ini menandakan bahwa model berhasil memahami pola dalam data training dengan baik.
3. **Validation Loss:** Loss validasi juga menunjukkan penurunan, meskipun terdapat fluktuasi kecil selama pelatihan. Fluktuasi ini dapat disebabkan oleh variasi dalam distribusi data validasi. Namun, tren keseluruhan menunjukkan bahwa loss validasi menurun, menunjukkan model berhasil meningkatkan performanya pada data yang belum pernah dilihat sebelumnya.

2. Analisis Training dan Validation Accuracy

Pada grafik "Model Accuracy" (kanan), terlihat peningkatan akurasi training dan validasi selama pelatihan:

1. **Awal Pelatihan:** Akurasi dimulai pada angka yang rendah, sekitar 20-30%, yang normal untuk tahap awal pelatihan.
2. **Peningkatan Akurasi:** Akurasi training meningkat secara konsisten hingga mencapai nilai mendekati 80% pada akhir pelatihan. Akurasi validasi juga menunjukkan peningkatan hingga sekitar 60-70%. Hal ini menunjukkan bahwa model

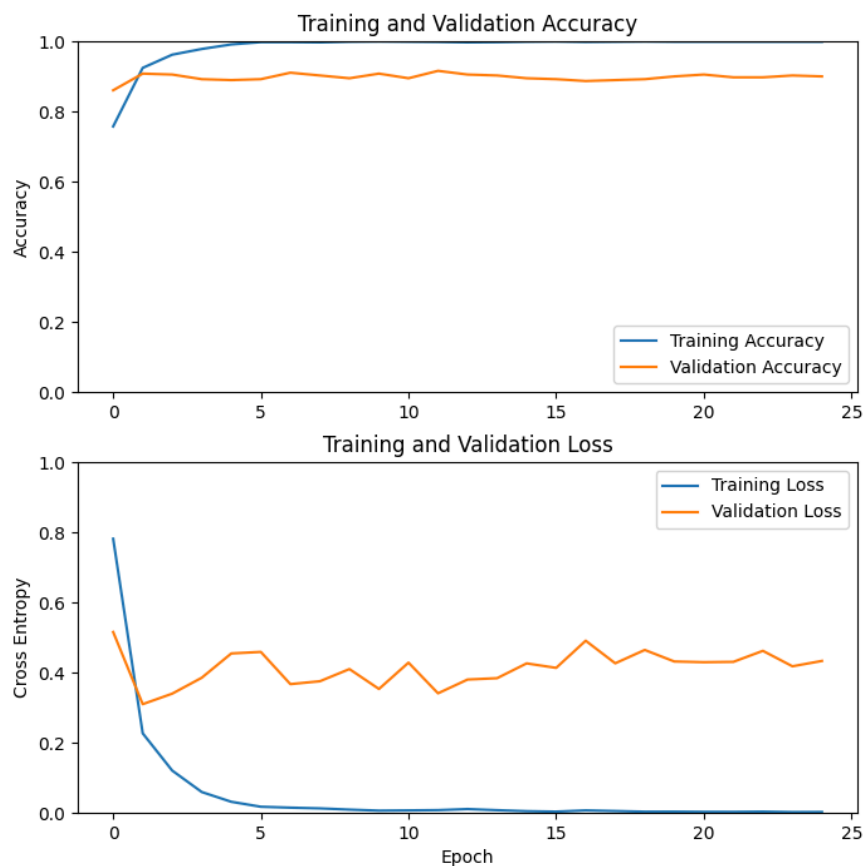
mampu belajar pola dalam data training sekaligus mempertahankan kemampuan generalisasi pada data validasi.

3. **Perbedaan Akurasi Training dan Validasi:** Perbedaan kecil antara akurasi training dan validasi dapat diindikasikan sebagai potensi overfitting minor. Meskipun demikian, selisih ini tidak terlalu besar, menunjukkan bahwa model masih memiliki generalisasi yang baik.

3. Potensi Penggunaan Model

Dengan akurasi training mendekati 80% dan akurasi validasi sekitar 60-70%, model ini sudah memiliki kinerja yang cukup baik untuk tugas klasifikasi tertentu. Namun, untuk dataset yang lebih kompleks atau aplikasi real-world, model ini masih bisa ditingkatkan lebih lanjut dengan metode yang telah disebutkan di atas. Grafik ini memberikan gambaran yang jelas tentang proses pelatihan model, menunjukkan pola belajar yang stabil dan potensi yang menjanjikan untuk penggunaan lebih lanjut.

Model 2 MobileNetV2



1. Grafik Akurasi (Training and Validation Accuracy)

1. **Kenaikan Akurasi Training:** Grafik akurasi pelatihan menunjukkan peningkatan yang cepat pada awal pelatihan (epoch 1-5) dan kemudian stabil di angka yang

tinggi mendekati 0.95-1.0. Hal ini menunjukkan bahwa model mampu belajar pola dari data pelatihan dengan baik.

2. **Konsistensi Akurasi Validasi:** Akurasi validasi mencapai puncaknya lebih awal dan stabil di sekitar 0.85. Stabilitas ini mengindikasikan bahwa model tidak mengalami overfitting, meskipun akurasi pelatihan lebih tinggi dibandingkan akurasi validasi.
3. **Perbedaan Akurasi Training dan Validasi:** Selisih kecil antara akurasi pelatihan dan validasi menunjukkan bahwa model telah berhasil menggeneralisasi data dengan cukup baik tanpa menunjukkan tanda-tanda overfitting yang signifikan.

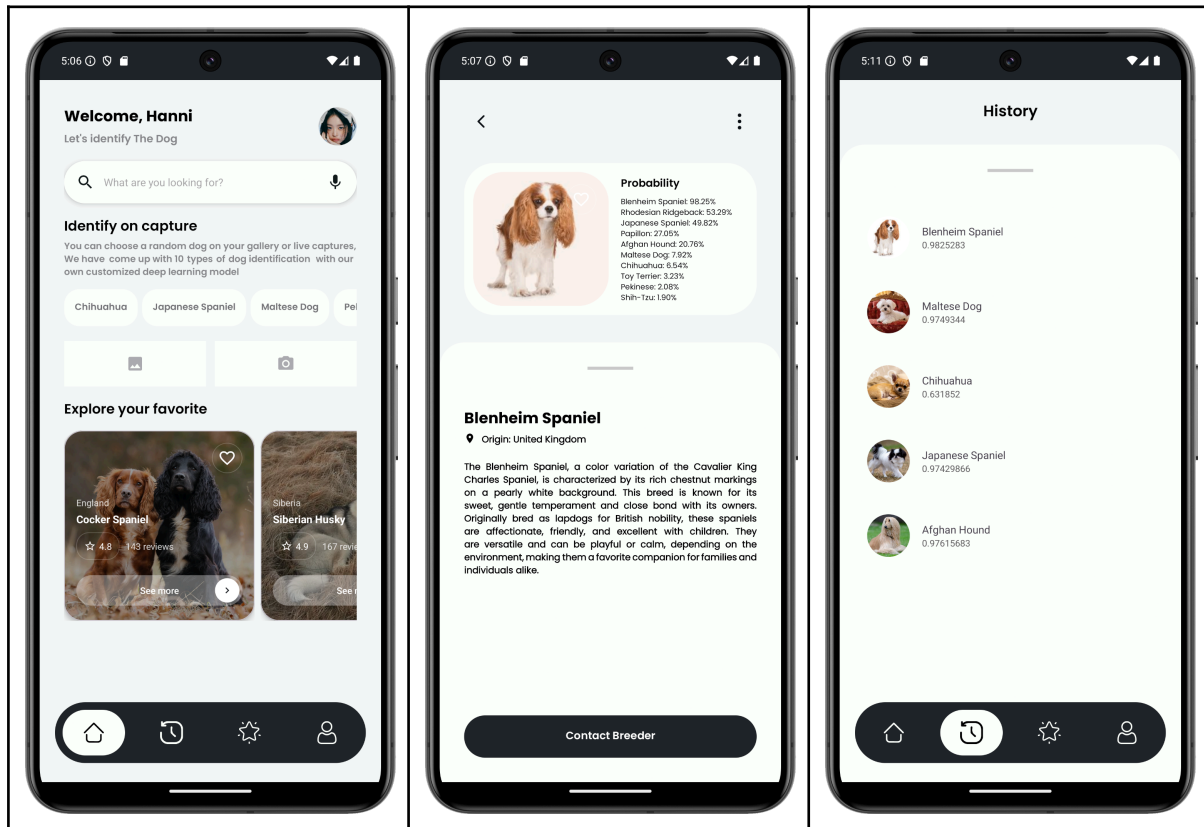
2. Grafik Loss (Training and Validation Loss)

1. **Penurunan Training Loss:** Loss pada data pelatihan menunjukkan penurunan yang drastis di awal pelatihan, kemudian terus menurun hingga mendekati nilai 0. Hal ini sesuai dengan tren pada akurasi pelatihan yang meningkat, menandakan bahwa model belajar secara efisien pada data pelatihan.
2. **Stabilitas Validation Loss:** Loss validasi awalnya menunjukkan fluktuasi kecil di beberapa epoch pertama sebelum akhirnya stabil pada nilai yang rendah. Stabilitas ini mencerminkan kemampuan model untuk memprediksi data validasi dengan tingkat kesalahan yang kecil.
3. **Keselaran Loss Training dan Validasi:** Perbedaan yang kecil antara loss pelatihan dan loss validasi menunjukkan bahwa model telah mencapai keseimbangan antara bias dan variansi, menghasilkan performa generalisasi yang baik.

DESIGN

Mobile App

Fitur yang ada pada Mobile App selain dengan mengklasifikasikan jenis anjing dalam input foto dari gallery dan pengambilan gambar langsung, hasil yang sudah ada akan langsung di save pada history layout yang bisa diakses melalui bottom navbar. Penyimpanan dilakukan secara lokal menggunakan konfigurasi Room tanpa memerlukan Backend.



AI Dog Breed Classifier

UPLOAD IMAGE



PREDICT

SHOW POSSIBLE BREEDS

Prediction Result

Predicted Breed: Maltese dog
Confidence: 0.99%

Maltese adalah sejenis anjing kecil dalam kategori anjing mainan. Nama anjing ini yang berarti 'dari Malta' dalam bahasa Inggris, biasanya tidak diterjemahkan dalam bahasa Indonesia [2] Salah satu ciri khas anjing Maltese ialah bahwa bulunya tidak rontok, bulunya lembut seperti sutra. Keturunan anjing ini berasal dari Kawasan Mediterania Tengah. Nama Maltese ini berasal dari pulau Mediterania pulau Malta, tetapi, nama ini teradang diartikan dengan mengacu ke pulau Adriatic, atau sebuah kota mati bernama Melita Sicilian.



© 2024 Dog Breed Classifier. All Rights Reserved.

CONCLUSION

Dari evaluasi terhadap empat model klasifikasi breed anjing yang dikembangkan untuk aplikasi web dan mobile, dapat disimpulkan bahwa model yang dibangun menggunakan scratch memiliki performa yang lebih rendah dibandingkan model yang menggunakan transfer learning. Model scratch untuk aplikasi web (dengan PyTorch) menunjukkan akurasi pelatihan sebesar 63% dan akurasi validasi hanya 48%, dengan indikasi overfitting yang signifikan setelah epoch ke-30. Sementara itu, model scratch untuk aplikasi mobile mencapai akurasi pelatihan mendekati 80% dan akurasi validasi di kisaran 60-70%, menunjukkan potensi overfitting minor tetapi masih memiliki kemampuan generalisasi yang cukup baik. Di sisi lain, model yang menggunakan transfer learning memberikan hasil yang jauh lebih baik. Model ResNetV2 untuk aplikasi web mencatat akurasi validasi sebesar 86%, dengan pelatihan yang efisien hanya dalam 6 epoch, meskipun sempat mengalami overfitting pada awal pelatihan. Model MobileNetV2 untuk aplikasi mobile mencapai akurasi pelatihan hampir 95-100% dan akurasi validasi stabil di angka 85%, menunjukkan keseimbangan yang baik antara bias dan variansi tanpa tanda-tanda overfitting yang signifikan.

Perbandingan antara kedua pendekatan ini menunjukkan bahwa model transfer learning jauh lebih unggul dalam hal akurasi, efisiensi, dan kemampuan generalisasi. Model scratch, meskipun memberikan wawasan mendalam tentang mekanisme deep learning, kurang optimal untuk aplikasi nyata karena memerlukan banyak upaya tambahan untuk meningkatkan performanya, seperti regularisasi, data augmentation, dan tuning hyperparameters. Sebaliknya, model transfer learning lebih cocok untuk digunakan dalam aplikasi nyata yang memerlukan akurasi tinggi, khususnya untuk tugas klasifikasi kompleks seperti ini. Dengan demikian, meskipun membangun model dari awal memberikan nilai pembelajaran yang besar, penggunaan transfer learning lebih disarankan untuk mencapai performa yang optimal dalam implementasi nyata.